

Instituto Tecnológico de las Américas (ITLA)
Tecnólogo en Desarrollo de Software

PROYECTO FINAL

Programación III

BibliotLA — Sistema de Gestión de Biblioteca
Aplicación web desarrollada con metodología Agile-Scrum

Sustentante: Triana Olivadia García de Jesús
Matrícula: 20231395
Asignatura: Programación III
Facilitador: Kelyn Tejada
Fecha de entrega: 17 de agosto de 2026

Santo Domingo, República Dominicana

Índice

1. Introducción

2. Estrategia de trabajo (planificación)

- 2.1. Nombre del proyecto de software
- 2.2. Tecnología a aplicar
- 2.3. Objetivo del proyecto
- 2.4. Alcance del proyecto
- 2.5. Cronograma del proyecto
- 2.6. Definición del primer Release
- 2.7. Requerimientos funcionales y no funcionales

3. Metodología Scrum

- 3.1. Tareas a ejecutar
- 3.2. Equipo de trabajo
- 3.3. Herramientas de gestión
- 3.4. Épicas
- 3.5. Ceremonias de Scrum
- 3.6. Historias de usuario

4. Plan de pruebas

- 4.1. Requerimientos y matriz de trazabilidad
- 4.2. Criterios de aceptación y de rechazo
- 4.3. Herramientas de pruebas y su justificación
- 4.4. Cronograma de ejecución de pruebas
- 4.5. Plantilla de casos de prueba
- 4.6. Equipo de pruebas y responsabilidades
- 4.7. Plan de automatización de pruebas
- 4.8. Ejecución de las pruebas y evidencias

5. Demostración y entregables

6. Conclusiones

7. Bibliografía

1. Introducción

Este documento recoge el desarrollo completo de BiblioITLA, un sistema de gestión de biblioteca construido como Proyecto Final de la asignatura Programación III. El trabajo no se limita a programar una aplicación: documenta cómo se planificó, cómo se organizó con Scrum, cómo se probó y qué resultados arrojaron esas pruebas.

El problema de partida es concreto. Una biblioteca académica que controla sus préstamos en hojas de cálculo depende de que el personal recuerde las políticas: cuántos libros puede llevarse cada socio, cuándo vence un préstamo y cuánta mora corresponde cobrar. Ese control manual falla precisamente cuando hay más movimiento. La propuesta consiste en trasladar esas reglas al software, de modo que un préstamo que incumple una política sea sencillamente imposible de registrar.

El proyecto se organizó en tres sprints de dos semanas siguiendo el marco Scrum, con diez historias de usuario repartidas en cuatro épicas. El primer Release cubre el circuito completo del negocio, desde el inicio de sesión hasta el cobro de la mora.

El apartado de mayor peso es el plan de pruebas. Se construyó una batería de 99 pruebas automatizadas en tres niveles —unitarias, de integración y de extremo a extremo con navegador real— que se ejecutan automáticamente ante cada cambio del código. Ese trabajo no es decorativo: detectó dos defectos reales antes de la entrega, documentados en el apartado 4.8.

El documento sigue el orden en que se hizo el trabajo: primero la planificación, luego la organización con Scrum, después el plan de pruebas con sus resultados y finalmente las conclusiones sobre lo que funcionó y lo que quedó pendiente.

2. Estrategia de trabajo (planificación)

2.1. Nombre del proyecto de software

BiblioITLA — Sistema de Gestión de Biblioteca

El nombre combina la actividad del sistema (biblioteca) con la institución para la que se plantea (ITLA). Es corto, se pronuncia con facilidad y describe sin ambigüedad de qué trata el producto.

2.2. Tecnología a aplicar

La selección buscó herramientas maduras, con buena documentación y que permitieran automatizar las pruebas sin infraestructura adicional. La última columna explica por qué se eligió cada una.

Tabla 1. Tecnologías seleccionadas y justificación.

Componente	Tecnología	Justificación
Lenguaje	Python 3.11	Sintaxis clara, ecosistema maduro de pruebas y es el lenguaje trabajado durante la asignatura.
Framework web	FastAPI 0.141	Genera documentación interactiva automáticamente, valida los datos de entrada con Pydantic y trae un cliente de pruebas integrado.
ORM	SQLAlchemy 2.0	Permite escribir las consultas en Python y cambiar de motor de base de datos sin reescribir la aplicación.
Base de datos	SQLite	No requiere instalar un servidor, lo que simplifica la evaluación. Al usar un ORM, migrar a PostgreSQL solo exige cambiar la cadena de conexión.
Plantillas	Jinja2	Renderizado de la interfaz en el servidor, sin la complejidad de un framework de frontend para un sistema de este tamaño.
Validación	Pydantic 2	Define los datos esperados de forma declarativa y rechaza automáticamente las peticiones mal formadas.
Pruebas unitarias y de API	pytest + pytest-cov	Sintaxis concisa, parametrización de casos y medición de cobertura.
Pruebas E2E	Playwright	Controla un navegador Chromium real, espera automáticamente a los elementos y graba video de cada ejecución.
Integración continua	GitHub Actions	Ejecuta las tres capas de pruebas en cada cambio, sin configurar servidores propios.
Control de versiones	Git y GitHub	Historial de cambios y alojamiento del repositorio de entrega.

2.3. Objetivo del proyecto

Objetivo general

Desarrollar una aplicación web que permita a una biblioteca académica administrar su catálogo, sus socios y el circuito completo de préstamos y devoluciones, sustituyendo el control manual en hojas de cálculo por un sistema que aplique automáticamente las políticas de la institución y ofrezca información actualizada sobre el estado del inventario.

Objetivos específicos

- Centralizar el catálogo de títulos y el control de ejemplares disponibles.
- Automatizar el cálculo de fechas de vencimiento y de la mora por retraso, eliminando los errores del cálculo manual.
- Impedir por diseño los préstamos que incumplen las políticas de la biblioteca, en lugar de depender de que el personal las recuerde.
- Ofrecer indicadores de gestión que permitan conocer el estado de la biblioteca sin elaborar reportes manuales.
- Construir el sistema con una batería de pruebas automatizadas que permita modificarlo con seguridad en futuras versiones.

2.4. Alcance del proyecto

Delimitar lo que el sistema no hace es tan importante como describir lo que hace: evita expectativas equivocadas y protege el alcance del Release durante el desarrollo.

El sistema incluye:

- Gestión del catálogo: registrar, consultar, buscar, actualizar y eliminar títulos, con control del número de ejemplares.
- Gestión de socios: registro, consulta, búsqueda y activación o desactivación.
- Circuito de préstamos: entrega, devolución, control de plazos y cálculo automático de la mora.
- Aplicación automática de las siete reglas de negocio de la biblioteca.
- Panel de indicadores: títulos, ejemplares, disponibles, préstamos activos y vencidos, socios activos y libros más prestados.
- Control de acceso mediante usuario y contraseña cifrada.
- API REST documentada de forma automática, para futuras integraciones.

El sistema NO incluye en este Release:

- Cobro en línea de la mora o integración con pasarelas de pago.
- Reservas de ejemplares y listas de espera.
- Envío de notificaciones por correo o SMS a los socios.
- Aplicación móvil nativa (la interfaz web es adaptable, pero no hay app).
- Catálogo público en línea para consulta de los estudiantes.
- Integración con el sistema académico del ITLA.
- Gestión multi-sucursal: el Release 1 asume una única biblioteca.

2.5. Cronograma del proyecto

Tabla 2. Cronograma de actividades, plazos y responsables.

#	Actividad	Inicio	Fin	Duración	Responsable
1	Levantamiento de requerimientos y definición del alcance	15/06/2026	19/06/2026	5 días	Product Owner
2	Diseño del modelo de datos y de la arquitectura	15/06/2026	19/06/2026	5 días	Equipo de desarrollo
3	Redacción de épicas e historias de usuario	17/06/2026	19/06/2026	3 días	Product Owner / Scrum Master
4	Sprint 1 — Catálogo de libros (HU-01, HU-02, HU-03)	22/06/2026	03/07/2026	10 días	Equipo de desarrollo
5	Sprint 2 — Circuito de préstamos (HU-04 a HU-07)	06/07/2026	17/07/2026	10 días	Equipo de desarrollo
6	Sprint 3 — Socios, panel y seguridad (HU-08, HU-09, HU-10)	20/07/2026	31/07/2026	10 días	Equipo de desarrollo
7	Automatización de pruebas y ejecución de la regresión	20/07/2026	07/08/2026	15 días	QA Automation
8	Pruebas de aceptación del Release 1	10/08/2026	12/08/2026	3 días	Product Owner / QA
9	Documentación final y grabación del video	13/08/2026	15/08/2026	3 días	Equipo completo
10	Entrega del Release 1	17/08/2026	17/08/2026	1 día	Scrum Master

2.6. Definición del primer Release

El primer Release entrega un sistema operativo de principio a fin para el trabajo diario del mostrador: el bibliotecario inicia sesión, consulta o amplía el catálogo, registra socios, entrega ejemplares con las políticas aplicadas automáticamente, recibe devoluciones con la mora ya calculada y revisa el estado general en un panel de indicadores. No es una maqueta: cubre el circuito completo del negocio, aunque deja fuera las funciones listadas como excluidas del alcance.

El Release 1 se compone de las diez historias de usuario del apartado 3.6, que suman 45 puntos de historia repartidos en tres sprints de dos semanas. Todas ellas están implementadas, probadas y demostradas en el video de entrega.

2.7. Requerimientos funcionales y no funcionales

Requerimientos funcionales

Tabla 3. Requerimientos funcionales del Release 1.

ID	Nombre	Descripción	Historia	Prioridad
RF-01	Consultar el catálogo	El sistema debe listar todos los títulos indicando ejemplares totales y disponibles en tiempo real.	HU-01	Alta
RF-02	Registrar y mantener libros	El sistema debe permitir crear, actualizar y eliminar títulos, validando que el ISBN sea único y que haya al menos un ejemplar.	HU-02	Alta
RF-03	Buscar y filtrar libros	El sistema debe permitir la búsqueda parcial por título, autor o ISBN, sin distinguir mayúsculas, y el filtrado por categoría.	HU-03	Media
RF-04	Calcular el vencimiento	El sistema debe fijar la fecha de vencimiento a 14 días naturales desde la entrega.	HU-04	Alta
RF-05	Registrar préstamos con validación	El sistema debe rechazar el préstamo cuando no haya ejemplares, el socio esté inactivo, haya alcanzado el límite de 3 préstamos o ya tenga ese mismo título sin devolver.	HU-04	Muy alta
RF-06	Registrar devoluciones y calcular la mora	El sistema debe cerrar el préstamo, reponer el ejemplar y calcular RD\$25 por cada día de retraso.	HU-05, HU-06	Muy alta
RF-07	Consultar préstamos por estado	El sistema debe permitir filtrar los préstamos entre activos, vencidos, devueltos y todos.	HU-07	Media
RF-08	Gestionar socios	El sistema debe permitir registrar y consultar socios con matrícula única y correo válido, y mostrar sus préstamos activos.	HU-09	Media
RF-09	Controlar el acceso	El sistema debe exigir autenticación para toda operación y almacenar las contraseñas cifradas.	HU-10	Alta
RF-10	Presentar indicadores de gestión	El sistema debe mostrar un panel con los indicadores del inventario y de los préstamos, y el ranking de libros más prestados.	HU-08	Media

Requerimientos no funcionales

Tabla 4. Requerimientos no funcionales y forma de verificarlos.

ID	Categoría	Descripción	Cómo se verifica
RNF-01	Rendimiento	Toda página o respuesta de la API debe entregarse en menos de 2 segundos con un catálogo de hasta 5.000 títulos.	Medición del tiempo de respuesta durante las pruebas E2E.
RNF-02	Seguridad de credenciales	Las contraseñas deben almacenarse cifradas con PBKDF2-HMAC-SHA256 y 120.000 iteraciones; nunca en texto plano.	Inspección de la base de datos y prueba automatizada de autenticación.
RNF-03	Control de acceso	Ninguna ruta del sistema, salvo el inicio de sesión, debe ser accesible sin una sesión válida.	Prueba E2E que solicita rutas protegidas sin sesión.
RNF-04	Usabilidad	Todo error debe explicar la causa concreta en lenguaje natural; no se aceptan códigos de error sin explicación.	Revisión de los mensajes en las pruebas E2E de casos negativos.
RNF-05	Mantenibilidad	La cobertura de pruebas del código de aplicación debe ser igual o superior al 80 %, y del 100 % en el módulo de reglas de negocio.	Informe de pytest-cov ejecutado en integración continua.
RNF-06	Portabilidad	El sistema debe ejecutarse en Windows, Linux y macOS sin cambios en el código.	Ejecución de la suite en el runner Ubuntu de GitHub Actions.
RNF-07	Compatibilidad	La interfaz debe funcionar en las versiones vigentes de Chrome, Edge y Firefox, y adaptarse a pantallas desde 360 px de ancho.	Pruebas E2E en Chromium y revisión manual del diseño adaptable.
RNF-08	Trazabilidad	Cada requerimiento funcional debe tener al menos una prueba automatizada que lo verifique.	Matriz de trazabilidad de la sección 3.1.

3. Metodología Scrum

El proyecto se organizó con Scrum en tres sprints de dos semanas. Se eligió este marco porque el circuito de préstamos se comprendió mejor al construirlo: trabajar por incrementos permitió incorporar reglas que no estaban en el planteamiento inicial sin rehacer el trabajo ya hecho.

3.1. Tareas a ejecutar

Las historias de usuario se descompusieron en veinte tareas técnicas. Una tarea es trabajo concreto para una persona, con una estimación en horas, mientras que una historia expresa valor para el usuario.

Tabla 5. Descomposición del trabajo en tareas técnicas.

ID	Tarea	Rol responsable	Sprint	Estimación
T-01	Diseñar el modelo entidad-relación y las tablas	Backend	Sprint 1	5 h
T-02	Configurar el proyecto, las dependencias y el repositorio	Backend	Sprint 1	3 h
T-03	Implementar los modelos SQLAlchemy y los datos iniciales	Backend	Sprint 1	4 h
T-04	Implementar el CRUD de libros en la capa de servicios	Backend	Sprint 1	6 h
T-05	Exponer la API REST del catálogo	Backend	Sprint 1	4 h
T-06	Construir la interfaz del catálogo y el buscador	Frontend	Sprint 1	6 h
T-07	Pruebas unitarias de validación de libros	QA	Sprint 1	3 h
T-08	Pruebas de API del catálogo	QA	Sprint 1	4 h
T-09	Aislar las reglas de negocio en funciones puras	Backend	Sprint 2	5 h
T-10	Implementar el registro de préstamos con sus validaciones	Backend	Sprint 2	7 h
T-11	Implementar la devolución y el cálculo de la mora	Backend	Sprint 2	5 h
T-12	Construir la pantalla de préstamos con filtros por estado	Frontend	Sprint 2	6 h
T-13	Pruebas unitarias de las reglas de negocio	QA	Sprint 2	6 h
T-14	Pruebas de API del circuito de préstamos	QA	Sprint 2	6 h
T-15	Implementar la gestión de socios	Backend	Sprint 3	5 h
T-16	Implementar la autenticación y el cifrado de contraseñas	Backend	Sprint 3	4 h
T-17	Construir el panel de indicadores y los reportes	Backend/Frontend	Sprint 3	6 h
T-18	Automatizar las pruebas E2E con Playwright	QA	Sprint 3	8 h
T-19	Configurar la integración continua en GitHub Actions	QA	Sprint 3	3 h
T-20	Redactar la documentación y grabar el video	Equipo	Sprint 3	6 h

3.2. Equipo de trabajo

Tabla 6. Roles, habilidades y responsabilidades.

Rol	Integrante	Habilidades requeridas	Responsabilidades
Product Owner	Triana O. García	Conocimiento del negocio bibliotecario, redacción de historias de usuario, priorización.	Definir y priorizar el Product Backlog, redactar los criterios de aceptación y aprobar el incremento en la Sprint Review.
Scrum Master	Triana O. García	Facilitación de reuniones, gestión de impedimentos, dominio del marco Scrum.	Organizar las ceremonias, retirar los obstáculos del equipo y velar por que se respete el marco de trabajo.
Desarrollador Backend	Triana O. García	Python, FastAPI, SQLAlchemy, diseño de bases de datos y de API REST.	Implementar el modelo de datos, las reglas de negocio, la capa de servicios y la API.
Desarrollador Frontend	Triana O. García	HTML5, CSS3, Jinja2, diseño adaptable y accesibilidad.	Construir la interfaz web, los formularios y la presentación de los mensajes de error.
QA Automation	Triana O. García	pytest, Playwright, diseño de casos de prueba, integración continua.	Diseñar el plan de pruebas, automatizar los tres niveles, mantener la regresión y reportar los defectos.

El Proyecto Final es un trabajo individual, de modo que la estudiante asume los cinco roles. La tabla no describe un equipo ficticio: describe las cinco responsabilidades que el proyecto exige y que, en un entorno profesional, se repartirían entre personas distintas. Separarlas por escrito obliga a cambiar de perspectiva conscientemente al escribir una historia, al programarla y al intentar romperla.

3.3. Herramientas de gestión

El seguimiento del trabajo se llevó en Jira Software, en un proyecto de tipo Scrum con su tablero, su Product Backlog y un Sprint Backlog por iteración. Las diez historias y las cuatro épicas están cargadas allí con sus criterios de aceptación y sus puntos de historia.

Se eligió Jira por ser el estándar en la industria y por distinguir de forma nativa entre épicas, historias y tareas, algo que un tablero genérico de tarjetas no hace. El repositorio incluye además el archivo `historias_usuario_jira.csv`, que permite reconstruir el tablero completo mediante la importación de datos de Jira.

Tabla 7. Herramientas de soporte al proyecto.

Herramienta	Uso en el proyecto
Jira Software	Product Backlog, Sprint Backlog, tablero y seguimiento de las historias.
GitHub	Alojamiento del repositorio e historial de cambios.
GitHub Actions	Integración continua: ejecuta las 99 pruebas ante cada cambio.
Visual Studio Code	Entorno de desarrollo.
Microsoft Teams	Ceremonias de Scrum y comunicación.
OneDrive institucional	Entrega del documento y del video.

3.4. Épicas

Las historias se agruparon en cuatro épicas según el área funcional que abordan. La épica del circuito de préstamos es la más grande porque concentra las reglas de negocio del sistema.

Tabla 8. Épicas del Release 1.

ID	Épica	Descripción	Historias	Puntos
EP-01	Gestión del catálogo	Alta, consulta, búsqueda y mantenimiento de títulos y ejemplares.	HU-01, HU-02, HU-03	11
EP-02	Gestión de socios	Registro y consulta de las personas autorizadas a tomar prestado.	HU-09	5
EP-03	Circuito de préstamos	Entrega, devolución, control de plazos y cálculo de mora. Es el núcleo del negocio y concentra las reglas más delicadas.	HU-04, HU-05, HU-06, HU-07	21
EP-04	Reportes y seguridad	Indicadores de gestión y control de acceso al sistema.	HU-08, HU-10	8

3.5. Ceremonias de Scrum

Las ceremonias se planificaron con fecha y hora fijas para los tres sprints: Sprint 1 del 22 de junio al 3 de julio, Sprint 2 del 6 al 17 de julio y Sprint 3 del 20 al 31 de julio de 2026.

Tabla 9. Calendario de ceremonias.

Ceremonia	Cuándo	Duración	Horario	Participantes	Propósito
Sprint Planning	Lunes de inicio de cada sprint (22/06, 06/07, 20/07)	2 horas	9:00 - 11:00	Equipo completo	Seleccionar las historias del sprint y descomponerlas en tareas.
Daily Stand-up	Todos los días laborables	15 minutos	9:00 - 9:15	Equipo de desarrollo	Qué se hizo ayer, qué se hará hoy y qué impedimentos existen.
Refinamiento del Backlog	Miércoles de la segunda semana (01/07, 15/07, 29/07)	1 hora	15:00 - 16:00	PO y equipo	Detallar y estimar las historias de los siguientes sprints.
Sprint Review	Viernes de cierre (03/07, 17/07, 31/07)	1 hora	14:00 - 15:00	Equipo y interesados	Demostrar el incremento funcionando y recoger retroalimentación.
Retrospectiva	Viernes de cierre (03/07, 17/07, 31/07)	45 minutos	15:15 - 16:00	Equipo completo	Qué funcionó, qué no y qué se cambia en el siguiente sprint.

3.6. Historias de usuario

Las diez historias siguen el formato «Como... quiero... para...», que obliga a expresar quién necesita la función y para qué, no solo qué hace el sistema. Cada una lleva sus criterios de aceptación y su estimación en puntos de historia según la sucesión de Fibonacci (3, 5, 8), donde el valor refleja la complejidad y no las horas de trabajo.

HU-04 es la historia de mayor puntuación (8) porque concentra las cuatro validaciones del negocio; HU-01, HU-03, HU-07 y HU-10 son las más sencillas (3). El total asciende a 45 puntos.

HU-01 — Consultar el catálogo de libros

Como bibliotecario quiero ver el listado completo de libros con sus ejemplares disponibles, para saber qué puedo prestar.

Épica	Sprint	Prioridad	Puntos de historia
EP-01	Sprint 1	Alta	3

Criterios de aceptación:

1. La tabla muestra ISBN, título, autor, categoría, ejemplares y disponibles.
2. Los disponibles se calculan como ejemplares menos préstamos activos.
3. Si el catálogo está vacío se muestra un mensaje informativo, no un error.
4. Un usuario sin sesión iniciada es redirigido al inicio de sesión.

HU-02 — Registrar un libro nuevo

Como bibliotecario quiero registrar libros nuevos, para mantener el catálogo actualizado.

Épica	Sprint	Prioridad	Puntos de historia
EP-01	Sprint 1	Alta	5

Criterios de aceptación:

1. El formulario solicita ISBN, título, autor, categoría, año y ejemplares.
2. El ISBN no puede repetirse; si ya existe se muestra un mensaje claro.
3. El número de ejemplares debe ser al menos 1.
4. Los espacios sobrantes al inicio y al final se eliminan automáticamente.
5. Tras guardar, el libro aparece en el listado con un aviso de confirmación.

HU-03 — Buscar libros en el catálogo

Como bibliotecario quiero buscar por título, autor o ISBN, para localizar un libro sin recorrer toda la lista.

Épica	Sprint	Prioridad	Puntos de historia
EP-01	Sprint 1	Media	3

Criterios de aceptación:

1. La búsqueda es parcial y no distingue mayúsculas de minúsculas.
2. Se puede combinar con un filtro por categoría.
3. Si no hay coincidencias se informa al usuario sin mostrar un error.
4. Un enlace permite limpiar los filtros y volver al listado completo.

HU-04 — Registrar un préstamo

Como bibliotecario quiero registrar la entrega de un ejemplar a un socio, para llevar el control de lo que está fuera de la biblioteca.

Épica	Sprint	Prioridad	Puntos de historia
EP-03	Sprint 2	Muy alta	8

Criterios de aceptación:

1. Se selecciona el socio y el libro desde listas desplegables.
2. La fecha de vencimiento se calcula automáticamente a 14 días.
3. Se rechaza el préstamo si no quedan ejemplares disponibles.
4. Se rechaza si el socio está inactivo.
5. Se rechaza si el socio ya tiene 3 préstamos activos.
6. Se rechaza si el socio ya tiene ese mismo título sin devolver.
7. Cada rechazo indica el motivo concreto, no un error genérico.
8. Al registrarse, los ejemplares disponibles del libro bajan en uno.

HU-05 — Registrar una devolución

Como bibliotecario quiero registrar la devolución de un ejemplar, para devolverlo al inventario disponible.

Épica	Sprint	Prioridad	Puntos de historia
EP-03	Sprint 2	Muy alta	5

Criterios de aceptación:

1. El botón de devolver solo aparece en los préstamos activos.
2. Al devolver se registra la fecha del día.
3. El ejemplar vuelve a contarse como disponible inmediatamente.
4. Un préstamo ya devuelto no puede devolverse otra vez.
5. Tras la devolución, el socio puede volver a llevarse ese mismo título.

HU-06 — Calcular la mora por retraso

Como bibliotecario quiero que el sistema calcule la mora, para cobrar el monto correcto sin hacer cuentas a mano.

Épica	Sprint	Prioridad	Puntos de historia
EP-03	Sprint 2	Alta	5

Criterios de aceptación:

1. Devolver el día del vencimiento o antes no genera mora.
2. A partir del día siguiente se cobran RD\$25 por cada día de retraso.
3. La mora se muestra en el detalle del préstamo y en el aviso de la devolución.
4. La mora nunca puede ser un valor negativo.

HU-07 — Consultar los préstamos por estado

Como bibliotecario quiero filtrar los préstamos por estado, para revisar rápidamente los pendientes y los vencidos.

Épica	Sprint	Prioridad	Puntos de historia
EP-03	Sprint 2	Media	3

Criterios de aceptación:

1. Existen los filtros: activos, vencidos, devueltos y todos.
2. Un préstamo vencido se resalta visualmente.
3. Cada fila muestra socio, libro, fechas y mora.

HU-08 — Ver los indicadores en el panel

Como responsable de la biblioteca quiero ver indicadores al entrar, para conocer el estado general de un vistazo.

Épica	Sprint	Prioridad	Puntos de historia
EP-04	Sprint 3	Media	5

Criterios de aceptación:

1. Se muestran títulos, ejemplares, disponibles, préstamos activos, vencidos y socios activos.
2. Se listan los cinco libros más prestados.
3. Se listan los préstamos vencidos con su fecha.
4. Los indicadores se actualizan al registrar un préstamo o una devolución.

HU-09 — Registrar y consultar socios

Como bibliotecario quiero administrar los socios, para saber a quién estoy autorizado a prestar.

Épica	Sprint	Prioridad	Puntos de historia
EP-02	Sprint 3	Media	5

Criterios de aceptación:

1. El formulario solicita matrícula, nombre, correo y teléfono.
2. La matrícula debe ser única en el sistema.
3. El correo debe tener un formato válido.
4. El listado muestra cuántos préstamos activos tiene cada socio.
5. Se advierte visualmente cuando un socio alcanza el límite de préstamos.

HU-10 — Iniciar sesión y controlar el acceso

Como administrador quiero que el sistema exija credenciales, para que solo el personal autorizado registre movimientos.

Épica	Sprint	Prioridad	Puntos de historia
EP-04	Sprint 3	Alta	3

Criterios de aceptación:

1. Sin sesión iniciada, cualquier ruta redirige al inicio de sesión.
2. Las credenciales incorrectas muestran un mensaje que no revela si falló el usuario o la contraseña.
3. Las contraseñas se almacenan cifradas con PBKDF2, nunca en texto plano.
4. El botón de salir cierra la sesión y bloquea el acceso posterior.

Tabla 10. Resumen de historias y distribución por sprint.

ID	Historia	Épica	Sprint	Puntos
HU-01	Consultar el catálogo de libros	EP-01	Sprint 1	3
HU-02	Registrar un libro nuevo	EP-01	Sprint 1	5
HU-03	Buscar libros en el catálogo	EP-01	Sprint 1	3
HU-04	Registrar un préstamo	EP-03	Sprint 2	8
HU-05	Registrar una devolución	EP-03	Sprint 2	5
HU-06	Calcular la mora por retraso	EP-03	Sprint 2	5
HU-07	Consultar los préstamos por estado	EP-03	Sprint 2	3
HU-08	Ver los indicadores en el panel	EP-04	Sprint 3	5
HU-09	Registrar y consultar socios	EP-02	Sprint 3	5
HU-10	Iniciar sesión y controlar el acceso	EP-04	Sprint 3	3
	TOTAL			45

4. Plan de pruebas

Este plan define qué se prueba, cómo se decide si el resultado es aceptable, con qué herramientas, quién lo ejecuta y cuándo. Su objetivo no es demostrar que el sistema funciona, sino intentar que falle antes de que lo haga en producción.

4.1. Requerimientos y matriz de trazabilidad

Los requerimientos funcionales y no funcionales se listaron en el apartado 2.7. La matriz siguiente los relaciona con la historia que los origina y con las pruebas que los verifican, de modo que ningún requerimiento quede sin comprobar.

Tabla 11. Matriz de trazabilidad requerimiento – historia – prueba.

Requerimiento	Historia	Pruebas que lo verifican	Nivel
RF-01	HU-01	test_devuelve_el_catalogo_inicial, test_lista_el_catalogo	API + E2E
RF-02	HU-02	TestCrearLibro (7 casos), test_registrar_un_libro	API + E2E
RF-03	HU-03	TestListarLibros (5 casos), test_buscar_por_titulo	API + E2E
RF-04	HU-04	TestCalcularVencimiento (7 casos)	Unitaria
RF-05	HU-04	TestValidarPrestamo (8), TestRegistrarPrestamo (11)	Unitaria + API + E2E
RF-06	HU-05, HU-06	TestCalcularMora (7), TestDevolucion (6), TestMoraPorRetraso	Unitaria + API
RF-07	HU-07	TestConsultarPrestamos (3 casos)	API
RF-08	HU-09	TestSocios (6), test_registrar_un_socio	API + E2E
RF-09	HU-10	TestAutenticacion (3), TestAcceso (4)	API + E2E
RF-10	HU-08	TestReportes (4), test_muestra_los_indicadores	API + E2E

4.2. Criterios de aceptación y de rechazo

Criterios de entrada (cuándo se puede empezar a probar)

- La historia de usuario tiene criterios de aceptación escritos y aprobados por el Product Owner.
- El código está integrado en la rama principal y compila sin errores.
- El entorno de pruebas está disponible con datos de ejemplo cargados.
- Los casos de prueba correspondientes están diseñados y revisados.

Criterios de aceptación (cuándo se aprueba el Release)

- El 100 % de las pruebas unitarias y de API terminan en verde.
- El 100 % de las pruebas E2E de los flujos principales terminan en verde.
- La cobertura del código de aplicación es igual o superior al 80 %, y del 100 % en el módulo de reglas de negocio.
- No queda ningún defecto abierto de severidad crítica o alta.

- Cada criterio de aceptación de cada historia tiene al menos una prueba automatizada que lo verifica.

Criterios de rechazo (cuándo se devuelve a desarrollo)

- Falla al menos una prueba de las reglas de negocio: el sistema podría prestar libros incumpliendo las políticas de la biblioteca.
- Existe un defecto crítico que impide completar el circuito de préstamo o devolución.
- Se detecta pérdida o corrupción de datos.
- Las contraseñas se almacenan sin cifrar o una ruta protegida es accesible sin sesión.
- La cobertura cae por debajo del 80 %.
- Una prueba resulta intermitente (pasa unas veces y falla otras) sin causa identificada: se trata como defecto porque destruye la confianza en la suite.

Criterios de suspensión (cuándo se detienen las pruebas)

- El entorno de pruebas no está disponible o la aplicación no arranca.
- Más del 30 % de los casos falla en la primera ejecución, lo que indica un problema de fondo y no defectos aislados.
- Se detecta un defecto bloqueante que impide continuar con el resto del ciclo.

4.3. Herramientas de pruebas y su justificación

Tabla 12. Herramientas de pruebas y motivo de su elección.

Herramienta	Se usa para	Justificación de la elección
pytest	Unitarias y de integración	Es el estándar de facto en Python. Permite parametrizar un mismo caso con muchos datos, lo que evita repetir código: las siete variantes del cálculo de mora se escriben una sola vez. Sus mensajes de fallo muestran los valores reales comparados.
pytest-cov	Medición de cobertura	Indica qué líneas nunca se ejecutan durante las pruebas. Se eligió sobre la medición manual porque muestra exactamente las líneas descubiertas, no solo un porcentaje, y se integra en la validación automática.
TestClient de FastAPI (httpx)	Pruebas de API	Ejecuta peticiones HTTP reales contra la aplicación sin levantar un servidor ni ocupar un puerto. Cada prueba de API tarda milisegundos, de modo que las 56 pruebas de integración corren en pocos segundos.
Playwright	Pruebas E2E en navegador	Controla un Chromium real. Se eligió sobre Selenium por su espera automática de elementos, que elimina la principal causa de pruebas intermitentes, y porque graba video y capturas sin configuración extra: esa grabación es la evidencia de automatización de este documento.
GitHub Actions	Integración continua	Ejecuta las tres capas en cada cambio subido al repositorio. Sin esto, las pruebas dependerían de que alguien recuerde ejecutarlas.

4.4. Cronograma de ejecución de pruebas

Tabla 13. Cronograma de pruebas manuales y automatizadas.

Actividad	Inicio	Fin	Tipo	Responsable
Diseño de los casos de prueba	22/06/2026	26/06/2026	Manual	QA Automation
Pruebas unitarias de reglas de negocio	29/06/2026	10/07/2026	Automatizada	QA Automation
Pruebas de API del catálogo	01/07/2026	03/07/2026	Automatizada	QA Automation
Pruebas de API del circuito de préstamos	13/07/2026	17/07/2026	Automatizada	QA Automation
Pruebas exploratorias de la interfaz	20/07/2026	24/07/2026	Manual	Product Owner
Automatización de las pruebas E2E	27/07/2026	31/07/2026	Automatizada	QA Automation
Pruebas de regresión completas	03/08/2026	07/08/2026	Automatizada	Integración continua
Pruebas de aceptación del Release 1	10/08/2026	12/08/2026	Mixta	Product Owner / QA
Regresión final y cierre del informe	13/08/2026	15/08/2026	Automatizada	QA Automation

4.5. Plantilla de casos de prueba

Se adoptó la plantilla siguiente, basada en la norma IEEE 829, para documentar cada caso de forma uniforme. Los campos son los mínimos que permiten a otra persona reproducir la prueba sin preguntar nada.

Tabla 14. Plantilla estándar para documentar un caso de prueba.

Campo	Descripción del campo
ID del caso	Identificador único, formato CP-000.
Historia asociada	Historia de usuario que origina el caso.
Título	Qué se comprueba, en una frase.
Precondición	Estado del sistema y datos necesarios antes de empezar.
Pasos	Acciones numeradas, en el orden exacto de ejecución.
Resultado esperado	Qué debe ocurrir, en términos observables.
Tipo	Unitaria, API, E2E; manual o automatizada.
Resultado obtenido	Superado o fallido, con la fecha de ejecución.

A continuación se documentan ocho casos representativos con la plantilla ya rellena. Se eligieron los que cubren las reglas de negocio más delicadas y los casos negativos, que son los que realmente ponen a prueba el sistema.

CP-001 — Registrar un préstamo válido

Campo	Contenido
ID del caso	CP-001
Historia asociada	HU-04
Título	Registrar un préstamo válido
Precondición	Existe el socio «Triana O. García» activo y el libro «Clean Code» con 3 ejemplares disponibles. Hay sesión iniciada.
Pasos	1. Entrar a la pantalla de Préstamos. 2. Seleccionar el socio en la lista desplegable. 3. Seleccionar el libro en la lista desplegable. 4. Pulsar «Registrar préstamo».
Resultado esperado	Se muestra el aviso de confirmación, el préstamo aparece en la lista de activos con vencimiento a 14 días y los ejemplares disponibles del libro bajan de 3 a 2.
Tipo	E2E automatizada
Resultado obtenido	Superado (15/08/2026)

CP-002 — Rechazar el préstamo de un título sin ejemplares

Campo	Contenido
ID del caso	CP-002
Historia asociada	HU-04
Título	Rechazar el préstamo de un título sin ejemplares
Precondición	El libro «Domain-Driven Design» tiene un solo ejemplar y ya está prestado a otro socio.
Pasos	1. Entrar a la pantalla de Préstamos. 2. Seleccionar un socio distinto al que tiene el ejemplar. 3. Seleccionar «Domain-Driven Design». 4. Pulsar «Registrar préstamo».
Resultado esperado	El sistema no registra el préstamo y muestra el mensaje «No hay ejemplares disponibles de este libro». La API responde 409.
Tipo	API automatizada
Resultado obtenido	Superado (15/08/2026)

CP-003 — Rechazar el cuarto préstamo simultáneo de un socio

Campo	Contenido
ID del caso	CP-003
Historia asociada	HU-04
Título	Rechazar el cuarto préstamo simultáneo de un socio
Precondición	El socio tiene exactamente 3 préstamos activos.
Pasos	1. Intentar registrar un cuarto préstamo para ese socio.
Resultado esperado	El sistema lo rechaza indicando que se alcanzó el máximo de 3 préstamos activos. El número de préstamos del socio sigue siendo 3.
Tipo	E2E automatizada
Resultado obtenido	Superado (15/08/2026)

CP-004 — Calcular la mora de una devolución con retraso

Campo	Contenido
ID del caso	CP-004
Historia asociada	HU-06
Título	Calcular la mora de una devolución con retraso
Precondición	Existe un préstamo cuyo vencimiento fue hace 5 días.
Pasos	1. Entrar a la pantalla de Préstamos. 2. Pulsar «Devolver» en ese préstamo.
Resultado esperado	El préstamo se cierra con la fecha del día y registra una mora de RD\$125 (5 días × RD\$25).
Tipo	API automatizada
Resultado obtenido	Superado (15/08/2026)

CP-005 — No cobrar mora al devolver dentro del plazo

Campo	Contenido
ID del caso	CP-005
Historia asociada	HU-06
Título	No cobrar mora al devolver dentro del plazo
Precondición	Existe un préstamo con vencimiento futuro.
Pasos	1. Registrar la devolución del préstamo.
Resultado esperado	El préstamo se cierra con mora igual a 0.
Tipo	Unitaria y API
Resultado obtenido	Superado (15/08/2026)

CP-006 — Rechazar un ISBN duplicado

Campo	Contenido
ID del caso	CP-006
Historia asociada	HU-02
Título	Rechazar un ISBN duplicado
Precondición	Ya existe en el catálogo el libro con ISBN 978-0132350884.
Pasos	1. Entrar a la pantalla de Libros. 2. Rellenar el formulario con ese mismo ISBN. 3. Pulsar «Registrar libro».
Resultado esperado	El sistema no crea el libro y muestra «Ya existe un libro con el ISBN 978-0132350884». El total del catálogo no cambia.
Tipo	E2E automatizada
Resultado obtenido	Superado (15/08/2026)

CP-007 — Bloquear el acceso sin sesión iniciada

Campo	Contenido
ID del caso	CP-007
Historia asociada	HU-10
Título	Bloquear el acceso sin sesión iniciada
Precondición	No hay ninguna sesión abierta en el navegador.
Pasos	1. Solicitar directamente la dirección /libros.
Resultado esperado	El sistema redirige al inicio de sesión y no muestra ningún dato del catálogo.
Tipo	E2E automatizada
Resultado obtenido	Superado (15/08/2026)

CP-008 — Impedir la devolución de un préstamo ya cerrado

Campo	Contenido
ID del caso	CP-008
Historia asociada	HU-05
Título	Impedir la devolución de un préstamo ya cerrado
Precondición	Existe un préstamo devuelto previamente.
Pasos	1. Solicitar de nuevo la devolución de ese préstamo por la API.
Resultado esperado	El sistema responde 409 con el mensaje «Este préstamo ya fue devuelto» y no modifica el inventario.
Tipo	API automatizada
Resultado obtenido	Superado (15/08/2026)

4.6. Equipo de pruebas y responsabilidades

Tabla 15. Responsabilidades en el proceso de pruebas.

Rol	Responsable	Responsabilidades en las pruebas
QA Automation	Triana O. García	Diseñar los casos, automatizar los tres niveles, mantener la suite de regresión y reportar los defectos con sus pasos de reproducción.
Desarrollador Backend	Triana O. García	Escribir las pruebas unitarias de su propio código antes de darlo por terminado y corregir los defectos reportados.
Desarrollador Frontend	Triana O. García	Mantener los atributos data-test de la interfaz, que son los que hacen estables las pruebas E2E, y verificar el diseño adaptable.
Product Owner	Triana O. García	Ejecutar las pruebas de aceptación sobre los criterios de cada historia y decidir si el incremento se aprueba.
Integración continua	GitHub Actions	Ejecutar la suite completa en cada cambio y bloquear la integración si algo falla.

4.7. Plan de automatización de pruebas

La automatización se organizó en tres niveles siguiendo la pirámide de pruebas descrita por Fowler (2012): cuanto más abajo está una prueba, más rápida y estable es, y más de ellas conviene tener.

Tabla 16. Niveles de automatización implementados.

Nivel	Cantidad	Qué cubre	Herramienta	Velocidad	Detalle
Nivel 1 — Unitarias	30 pruebas	reglas.py (funciones puras)	pytest	Milisegundos	Cubren cada regla de negocio y sus casos límite: devolución anticipada, el mismo día del vencimiento, un día después, años bisiestos y cambios de mes. Al no tocar la base de datos, son inmediatas y deterministas.
Nivel 2 — Integración/API	56 pruebas	servicios, routers y base de datos	pytest + TestClient	Segundos	Recorren la pila completa por HTTP. Verifican los códigos de estado, la forma de las respuestas, las validaciones y que el inventario quede consistente tras cada operación.
Nivel 3 — E2E	13 pruebas	la aplicación completa en un navegador	Playwright + Chromium	Segundos por prueba	Reproducen lo que hace el bibliotecario: iniciar sesión, registrar, prestar, equivocarse y devolver. Son las más lentas y frágiles, por eso se limitan a los flujos principales.

Estrategia y decisiones tomadas

- Se sigue la pirámide de pruebas: muchas unitarias, bastantes de integración y pocas E2E. Invertir la pirámide produce suites lentas y frágiles que el equipo termina ignorando.
- Las reglas de negocio se aislaron en funciones puras precisamente para poder probarlas sin base de datos. Ese aislamiento no es un detalle de estilo: es lo que permite tener 30 pruebas que corren en milisegundos.

- Cada prueba parte de una base de datos nueva. Durante el desarrollo, dos pruebas E2E fallaron porque compartían estado con las anteriores; se corrigió reiniciando los datos antes de cada prueba, de modo que el resultado no dependa del orden de ejecución.
- La interfaz se localiza mediante atributos data-test y no por texto ni por posición, para que un cambio de redacción o de diseño no rompa las pruebas.
- Se automatiza todo lo repetitivo y objetivo. Quedan como pruebas manuales la valoración estética de la interfaz y las pruebas exploratorias, donde el criterio humano encuentra lo que nadie pensó en programar.
- La suite se ejecuta automáticamente en GitHub Actions ante cada cambio; si algo falla, el commit queda marcado en rojo.

4.8. Ejecución de las pruebas y evidencias

La suite completa se ejecutó el 15 de agosto de 2026 sobre el Release 1. Los resultados son los siguientes.

Tabla 17. Resultados de la última ejecución completa.

Nivel	Total	Superadas	Fallidas	Porcentaje
Pruebas unitarias	30	30	0	100 %
Pruebas de API	56	56	0	100 %
Pruebas E2E	13	13	0	100 %
Total	99	99	0	100 %

Cobertura de código

Tabla 18. Cobertura por módulo.

Módulo	Responsabilidad	Cobertura
app/reglas.py	Reglas de negocio	100 %
app/esquemas.py	Validación de datos	100 %
app/principal.py	Arranque de la aplicación	100 %
app/modelos.py	Entidades de datos	97 %
app/routers/api.py	API REST	96 %
app/servicios.py	Capa de servicios	95 %
app/base_datos.py	Conexión y datos iniciales	89 %
app/routers/web.py	Interfaz web	35 % (ver nota)
TOTAL		83 %

La cifra de `app/routers/web.py` merece una aclaración honesta. Ese módulo sí está probado: las 13 pruebas E2E recorren todas sus rutas con un navegador real. Ocurre que esas pruebas levantan el servidor en un proceso aparte y la herramienta de cobertura solo mide el proceso donde se ejecuta `pytest`, de modo que no contabiliza lo que sucede en el servidor. El 35 % es una limitación de la medición, no una falta de pruebas. Medirlo correctamente exige activar la cobertura en subprocesos, mejora prevista para el Release 2.

Defectos detectados por la automatización

Los dos defectos siguientes fueron encontrados por las pruebas automatizadas durante el desarrollo, no por un usuario. Se documentan porque son la evidencia concreta de que el plan de pruebas cumplió su función.

Tabla 19. Defectos detectados, causa y corrección.

ID	Severidad	Descripción	Causa raíz	Corrección	Estado
DEF-001	Alta	Dos pruebas E2E fallaban de forma intermitente al ejecutarse todas juntas.	Las pruebas compartían una única base de datos durante toda la sesión, de modo que una prueba heredaba los préstamos creados por la anterior.	Se añadió un reinicio automático de los datos antes de cada prueba.	Cerrado
DEF-002	Media	La aplicación no arrancaba al validar el correo de un socio.	Pydantic requiere el paquete email-validator para el tipo EmailStr y no estaba declarado entre las dependencias.	Se añadió pydantic[email] a requirements.txt.	Cerrado

Evidencias disponibles en el repositorio

- evidencias/demo/demo_biblioitla.mp4 — video del recorrido completo por el Release 1, grabado automáticamente con Playwright.
- evidencias/capturas/ — una captura de pantalla por cada prueba E2E, generada automáticamente al finalizar cada caso.
- evidencias/videos/ — grabación en video de cada prueba E2E ejecutándose.
- evidencias/cobertura/index.html — informe de cobertura navegable, línea por línea.
- Pestaña Actions del repositorio — historial de todas las ejecuciones de la suite en integración continua.

5. Demostración y entregables

El video de demostración recorre el incremento del Release 1 en el orden en que trabaja el bibliotecario: inicio de sesión, panel de indicadores, catálogo, alta de un libro, búsqueda, registro de un socio, préstamo, rechazo de un préstamo que incumple una regla, devolución y actualización de los indicadores.

Se incluye a propósito el intento de préstamo rechazado: mostrar solo el camino correcto no demuestra que las reglas de negocio estén implementadas.

Tabla 20. Enlaces de la entrega.

Entregable	Enlace
Repositorio del código fuente	https://github.com/USUARIO/biblioitla
Tablero de Jira con las historias de usuario	https://TU-DOMINIO.atlassian.net/jira/software/projects/BIB/boards/1
Código de las pruebas automatizadas	https://github.com/USUARIO/biblioitla/tree/main/pruebas
Integración continua (ejecuciones de la suite)	https://github.com/USUARIO/biblioitla/actions
Video de demostración del Release 1	Enlace de OneDrive institucional al video de demostración

Los enlaces se colocan además en la opción «Texto en línea» de la plataforma, como exige el reglamento de entrega.

6. Conclusiones

El proyecto entrega un sistema funcional de principio a fin, no una maqueta: el bibliotecario puede cubrir su trabajo diario completo, desde iniciar sesión hasta cobrar una mora, y las siete políticas de la biblioteca se aplican automáticamente.

La decisión técnica más rentable fue aislar las reglas de negocio en funciones puras, sin base de datos ni HTTP. Eso permitió escribir 30 pruebas unitarias que se ejecutan en milisegundos y cubren el módulo al 100 %, incluyendo casos límite como los años bisiestos o la devolución el mismo día del vencimiento, que difícilmente se probarían a mano.

Scrum resultó adecuado porque el circuito de préstamos se entendió mejor al construirlo. La regla que impide a un socio llevarse dos veces el mismo título no estaba en el planteamiento inicial: apareció al probar el Sprint 2 y se incorporó como criterio de aceptación de HU-04. Con un método en cascada, esa corrección habría llegado al final.

El plan de pruebas demostró su valor de forma concreta. Dos defectos reales (DEF-001 y DEF-002) fueron detectados por la automatización y no por un usuario. El primero es especialmente ilustrativo: unas pruebas que fallaban de forma intermitente revelaron que el aislamiento entre casos estaba mal diseñado, un problema que a mano habría pasado inadvertido.

La pirámide de pruebas se respetó de forma deliberada: 30 unitarias, 56 de integración y solo 13 E2E. Las E2E dan la mayor confianza pero son las más lentas y frágiles, por lo que se reservaron para los flujos principales en lugar de intentar cubrirlo todo con ellas.

Queda pendiente para el Release 2: medir correctamente la cobertura de la capa web, añadir reservas de ejemplares, notificaciones por correo y un catálogo público de consulta para los estudiantes.

7. Bibliografía

- Schwaber, K. y Sutherland, J. (2020). La Guía de Scrum: Las Reglas del Juego. Scrum.org. Recuperado de <https://scrumguides.org/>
- Cohn, M. (2004). User Stories Applied: For Agile Software Development. Addison-Wesley Professional.
- Cohn, M. (2009). Succeeding with Agile: Software Development Using Scrum. Addison-Wesley Professional.
- Crispin, L. y Gregory, J. (2009). Agile Testing: A Practical Guide for Testers and Agile Teams. Addison-Wesley Professional.
- Fowler, M. (2012). Test Pyramid. [martinfowler.com](https://martinfowler.com/bliki/TestPyramid.html). Recuperado de <https://martinfowler.com/bliki/TestPyramid.html>
- International Software Testing Qualifications Board (2018). Programa de Estudio de Nivel Básico ISTQB, versión 2018. ISTQB.
- IEEE (2008). IEEE Std 829-2008: Standard for Software and System Test Documentation. IEEE Computer Society.
- Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.
- Ramírez, S. (2026). FastAPI Documentation. Recuperado de <https://fastapi.tiangolo.com/>
- Microsoft (2026). Playwright Documentation. Recuperado de <https://playwright.dev/python/docs/intro>
- Krekel, H. y colaboradores (2026). pytest Documentation. Recuperado de <https://docs.pytest.org/>